Guide for Python Node configuration

This is a guide on how to configure a Python node in order to use a .onnx model to predict X and Y coordinates from an input coming from LabVIEW with a specific dimension.

1. Python Functions to use in LabVIEW

LabVIEW has three functions that are necessary to connect with a python code.

- Open Python Session 
- Python Node 
- Close Python Session 

The function in which we need to concentrate more is the Python Node, as this is the function in which we call a .py file and its function in order to get a specific output out of that code. As for the other two, these are simple to use and need just basic setup.

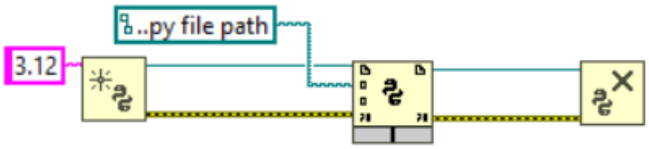
2. Python Functions initial configuration

- Open Python Session: For this function there is only one terminal to setup called Python version. To do so, we need to create a constant wired to this terminal and enter the version of python we will be using (eg. 3.12, 3.12.10, etc.)

- Python Node: First of all, it is necessary to wire the session and error outs from the Open Python Session to this function. After this, we need to create a constant wired to the module path terminal. Inside thai constant we will insert the path to our .py file. Further configuration will be explained later as it is dependent on how our model behaves.

- Close Python Session: For this function we only need to connect the sessions and error outs from the Python Node into this block.

After this first configuration, your blocks should look like this:



3. Python Node configuration

Now that we have the initial configuration, we need to configure our Python Node with the correct return type, call the desired function, and wire the necessary input and indicator for the outputs.

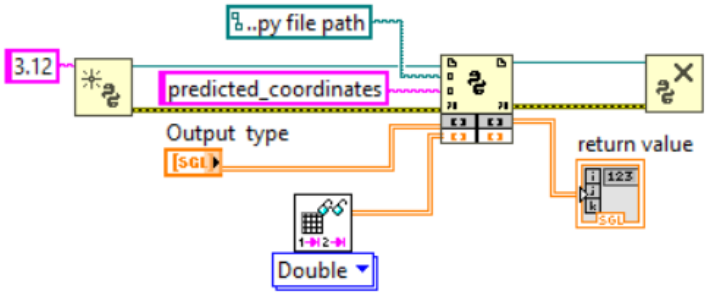
- Return Type: the gray terminal on the left is the return type which helps us establish how our output will look. It could be any type of data (integer, double, float) and have any shape (single point, array, cluster). For this exercise the form of our output is a X coordinate and a Y coordinate arranged in a 2D array. This was established in our model and .py. Because of this, we need to create an array controller to wire to the return type terminal. Make sure it is a 2D array, if not it will not work.

- Function Name: For this terminal we need to create a constant in which we will write the name of the function used to predict coordinates.

- Input Parameter: In order to get this terminal, we need to draw down the bottom of the block, which will create two white terminals. The one on the left is the Input Parameter terminal, and we need to wire the data we will use to predict coordinates to this terminal. For this example, we wired a .csv file reading, but ideally it will be an array with real time data collection.

- Return Value: For this terminal we simply need to create and wire an array indicator that will show the predicted values. It will eventually be connected to the calculation of error and PID in the real time loop control.

After adding this elements into the Python Node, your hole system should look something like this (reading csv block not configured, and could change in real application):



.py used for this exercise
NOTE: The used .py will probably change, as it depends on how the model works. In this case, it receives a 2000 X and Y dataset, and was trained under standardized data.

```
import numpy as np
import onnxruntime as ort
import os

# Get the directory where this script is located
script_dir = os.path.dirname(os.path.abspath(__file__))

# 1. Initialize the ONNX session (No PyTorch needed!)
onnx_path = os.path.join(script_dir, "zeus_laser_tracker.onnx")
session = ort.InferenceSession(onnx_path)

# 2. Load the scaling parameters generated by your standalone test
scaling_path = os.path.join(script_dir, "scaling_params.npz")
scaling_data = np.load(scaling_path)
mean = scaling_data['mean']  # [mean_x, mean_y]
std = scaling_data['std']    # [std_x, std_y]

def predict_coordinates(lv_array_2d):
    """
    lv_array_2d: Raw X and Y coordinates directly from LabVIEW
    """
    # 1. Reconstruct the 2D matrix layout from LabVIEW
    raw_flat = np.array(lv_array_2d, dtype=np.float32).flatten()
    reconstructed_2d = raw_flat.reshape(-1, 2)
    current_rows = reconstructed_2d.shape[0]

    repeats = int(np.ceil(2000 / current_rows))
    padded_data = np.tile(reconstructed_2d, (repeats, 1))
    truncated_data = padded_data[:2000, :]

    # 3. STANDARDIZE THE INCOMING DATA (Match how the model was trained!)
    standardized_data = (truncated_data - mean) / std

    # 4. Reshape to 3D Tensor format: (1, 2000, 2)
    input_tensor = np.expand_dims(standardized_data, axis=0).astype(np.float32)
```

```
# 5. Execute ONNX Inference
outputs = session.run(['predicted_xy'], {'input_timeline': input_tensor})

# 6. UN-STANDARDIZE THE PREDICTION BACK TO REAL RAW COORDINATES
scaled_pred = outputs[0][0]
real_x = float((scaled_pred[0] * std[0]) + mean[0])
real_y = float((scaled_pred[1] * std[1]) + mean[1])

# 7. Format perfectly as a 2D nested list for LabVIEW
prediction_2d = [[real_x, real_y]]

return prediction_2d
```